



Deep Learning in Asset Pricing

Replication and extensions

LSTM, Transformer, rolling performance and regimes

Reza Zamani, Hrafnhildur Lif Jonsdottir

Paraj Goyal and Elouan Bahri

Professor: Ali Kakhbod

UC Berkeley

Group 7 · October 2026

Presentation overview

The original paper

Economic pricing restrictions, hidden macro states and adversarial instruments.

Our replication

Published benchmarks, local model rankings and validation checks.

New results from Codes 08–11

Architecture, rolling windows, economic regimes and their combinations.

The original paper's research question

Estimate a pricing kernel

Learn the stochastic discount factor that prices assets consistently with no arbitrage.

Use a richer information set

Firm characteristics and macroeconomic history allow risk exposures to vary over time.

Separate pricing from return prediction

An economic moment objective guides the network toward pricing-relevant information.

How the model learns asset prices

1. Summarize the economic environment

LSTM networks read macroeconomic history and create hidden states for the pricing and adversarial branches.

2. Build the pricing portfolio

A feedforward network combines firm characteristics with the pricing states to assign stock weights and construct the SDF.

3. Challenge and improve the model

The adversary finds instruments that reveal large pricing errors. The pricing network learns to reduce those errors.

The original model architecture

Component	Input and role	Output
Macro SDF encoder	178 macro series, LSTM	4 states
Pricing-weight network	46 characteristics + states	Two layers of 64 units
Macro instrument encoder	178 macro series, LSTM	32 states
Conditional network	Characteristics + states	8 bounded instruments

The pricing objective links the SDF network and the adversarial instruments.

Data and chronological evaluation

Sample	Period	Months
Training	1967–1986	240
Validation	1987–1991	60
Test	1992–2016	300

46 firm characteristics and 178 macroeconomic inputs

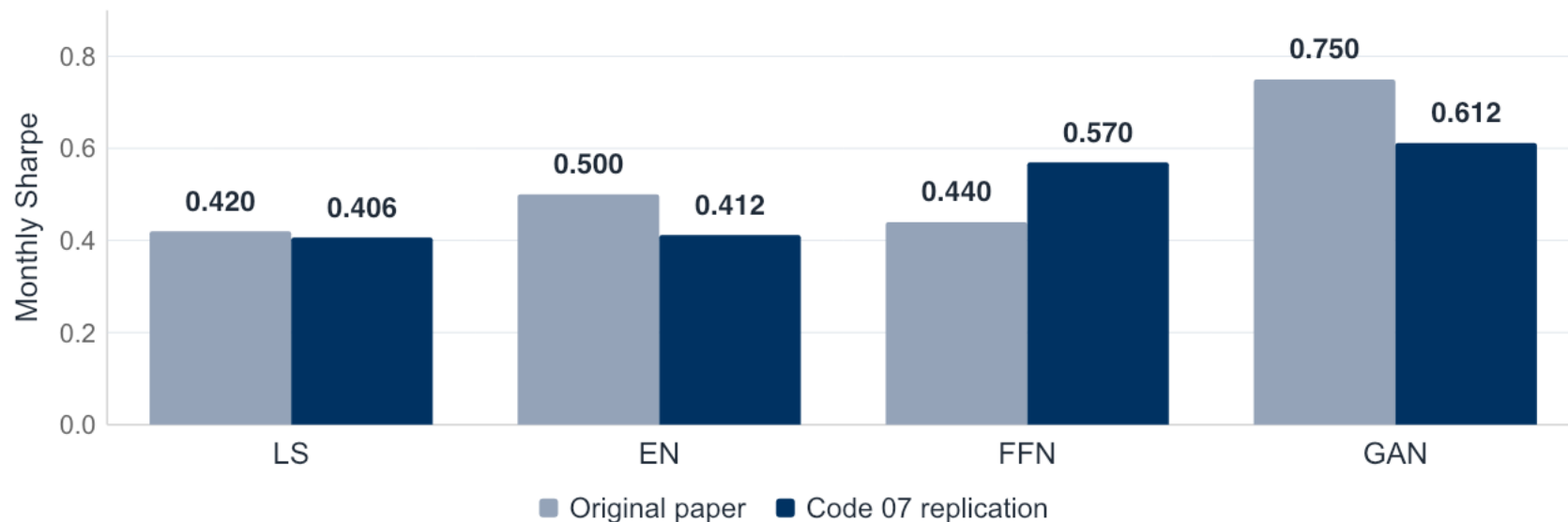
Fit on training months and select checkpoints using validation months.

The paper's published results

Model	Train	Validation	Test
LS	1.800	0.580	0.420
EN	1.370	1.150	0.500
FFN	0.450	0.420	0.440
GAN	2.680	1.430	0.750

GAN has the highest published monthly Sharpe in every split.

Replication versus the original paper



GAN stays first, while FFN and EN exchange test-period ranks.

Replication validation and common signals

Check	Our result	Reference
FF-3 test Sharpe	0.196	Paper 0.190
FF-5 test Sharpe	0.223	Paper 0.220
GAN test correlation	0.800	Authors' monthly GAN
Leading characteristics	ST_REV, SUV, r12_2	Same top three

These checks support the replication's economic similarity to the original.

Questions left by the previous presentations

Architecture

Would causal attention change the learned macro representation?

Time variation

Does one full-test Sharpe hide changes in relative performance?

Economic conditions

Do recession and volatility groups explain where the models differ?

What Codes 08–11 add

Code	Analysis	Question
08	LSTM versus Transformer	Which encoder works at a common budget?
09	36/60-month rolling evaluation	When does performance change?
10	Recession and volatility regimes	Where do return and risk differ?
11	Consolidation and combinations	How do the comparisons fit together?

The new contribution is a linked evaluation of architecture and economic conditions.

The completed experiment uses shorter training

Setting	Audited author setup	Our completed run
Stage 1 / 2 / 3 epochs	256 / 64 / 1024	16 / 4 / 48
Optimizer passes	4	4
Checkpoint warm-up	64 epochs	4 epochs
Seeds per architecture	9	9
Training framework	TensorFlow author code	PyTorch port

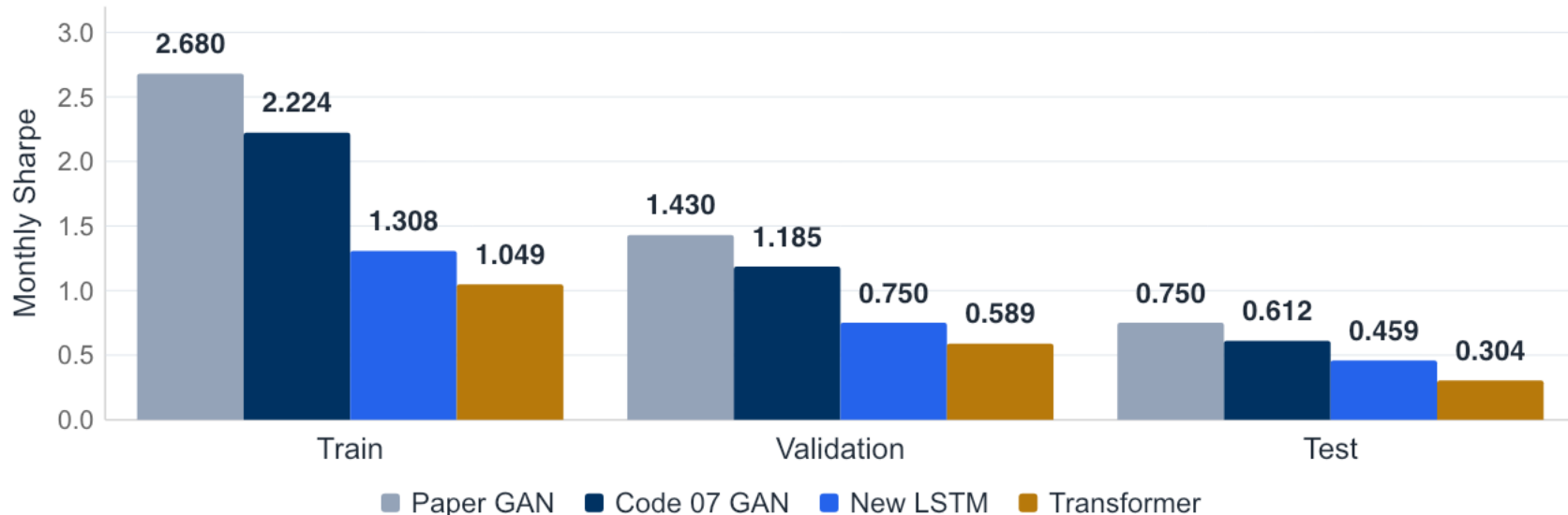
All 18 new models completed in 12.9 minutes. Full training remains a separate test.

LSTM and Transformer encoder comparison

Choice	LSTM	Transformer
Sequence processing	Recurrent state	Causal self-attention
Macro inputs	178	178
SDF / instrument states	4 / 32	4 / 32
Transformer specification	Not applicable	2 layers, 4 heads, width 32
Economic objective	Shared	Shared
Training seeds and schedule	Shared	Shared

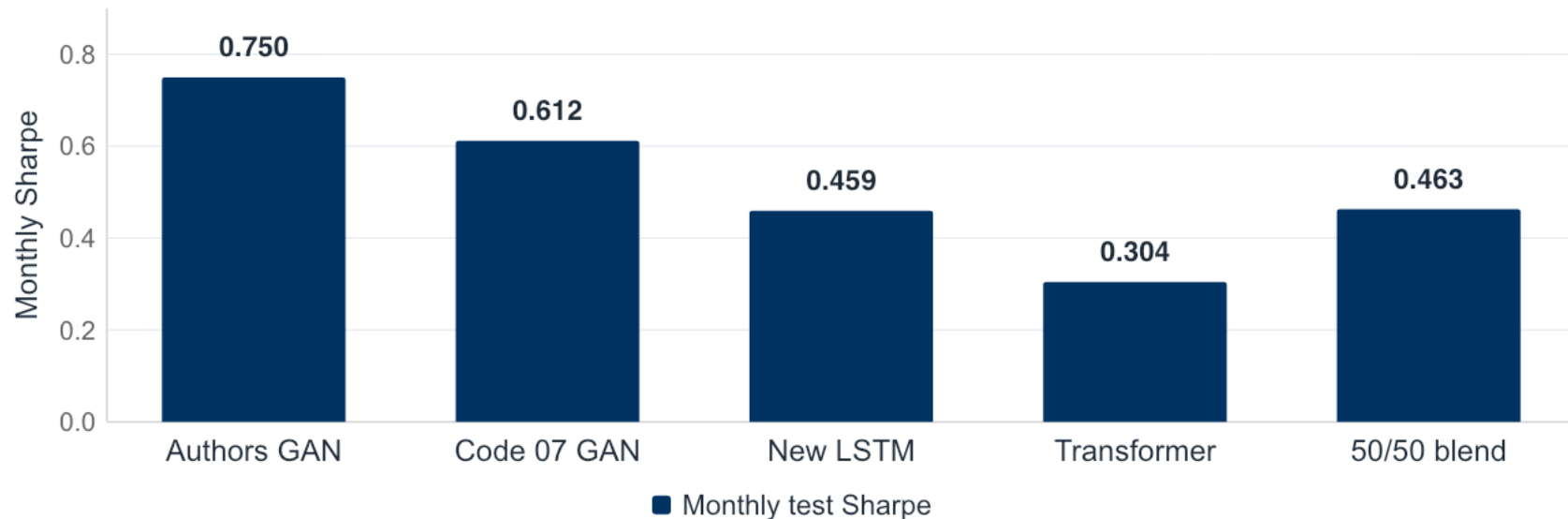
The comparison replaces both macro encoders and keeps the output dimensions.

Train, validation and test comparison



The new LSTM exceeds Transformer in all three splits under the reduced budget.

Full-test Sharpe across the five portfolios



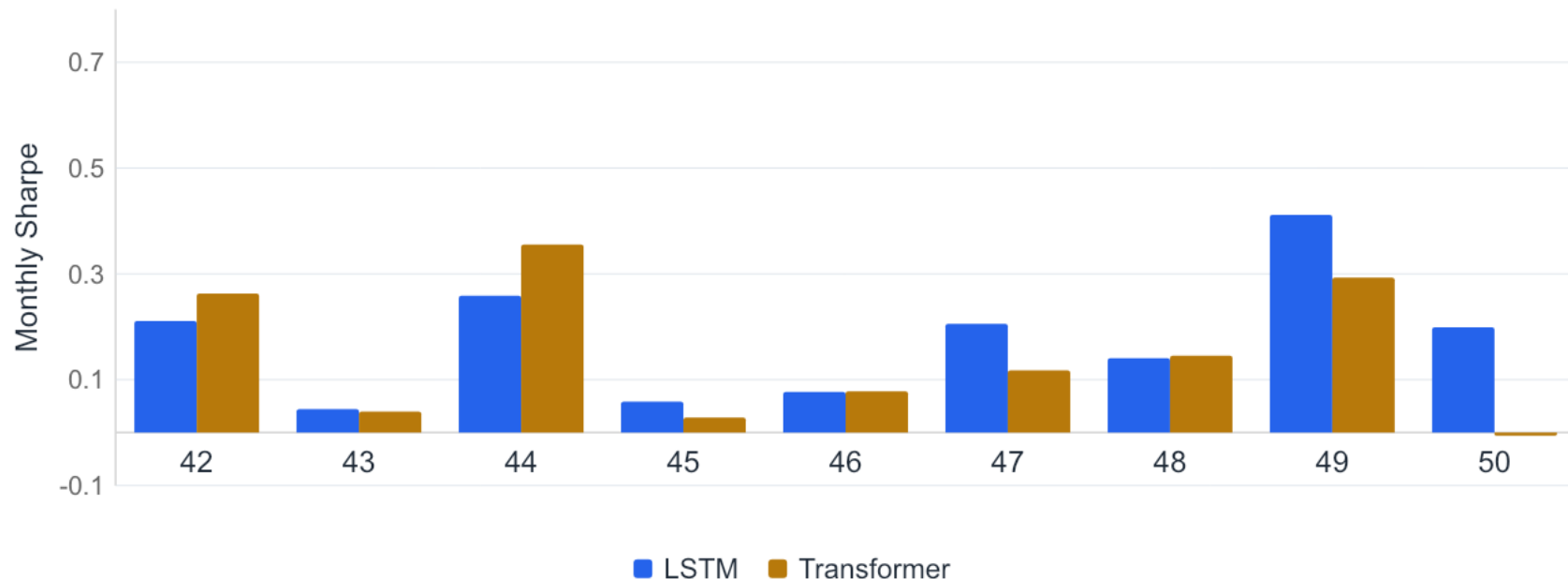
Author GAN 0.750, Code 07 0.612, new LSTM 0.459, Transformer 0.304.

Return and risk explain the architecture gap

Portfolio	Mean monthly	Annual volatility	Max drawdown
Authors GAN	0.78%	3.61%	-7.02%
Code 07 GAN	0.70%	3.96%	-6.51%
New LSTM	0.51%	3.87%	-11.14%
Transformer	0.42%	4.82%	-15.35%
50/50 blend	0.47%	3.50%	-6.57%

Transformer combines a lower mean return with greater volatility than new LSTM.

Initialization sensitivity across nine seeds



Ensemble Sharpe uses averaged raw weights, rather than the mean member Sharpe.

Rolling evaluation uses fixed saved models

Definition	36 months	60 months
Complete windows	265	241
First endpoint	December 1994	December 1996
Last endpoint	December 2016	December 2016
Statistic	Annualized rolling Sharpe	Annualized rolling Sharpe

Each window reuses earlier returns. Longer windows smooth short episodes.

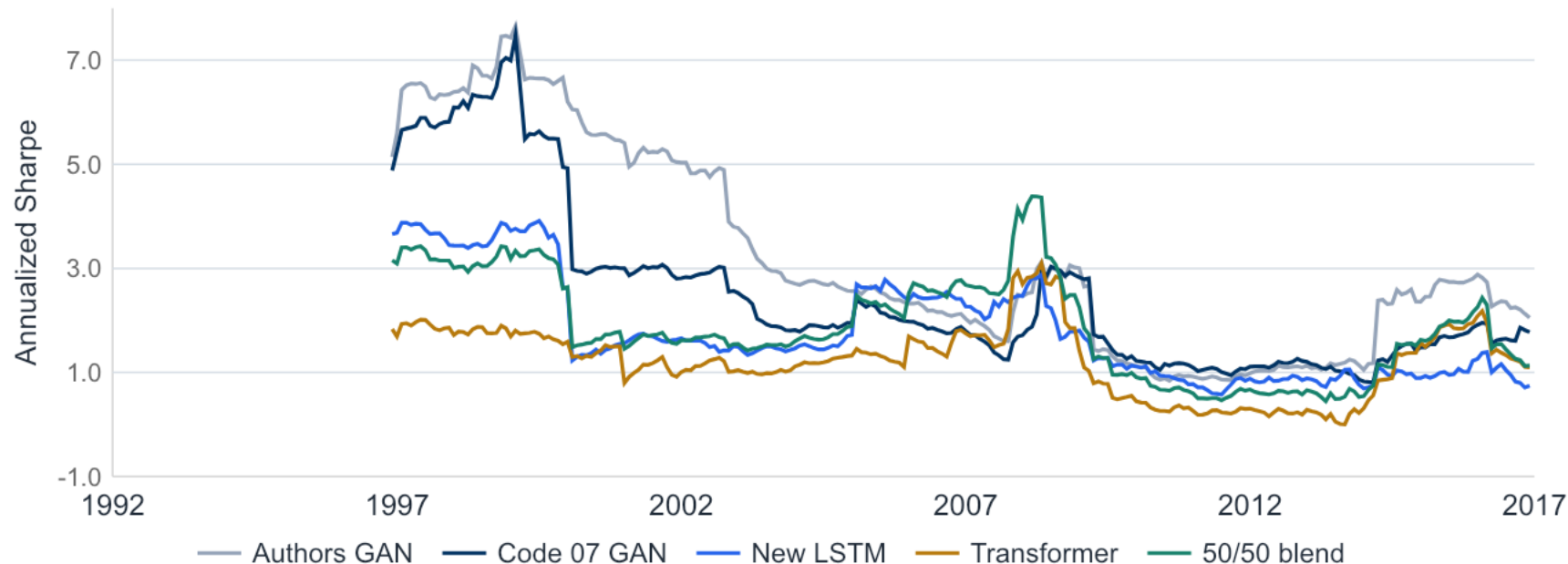
This experiment evaluates fixed models. It does not refit an expanding-window model.

36-month rolling Sharpe



Transformer beats new LSTM in 36.6% of complete windows.

60-month rolling Sharpe



Transformer beats new LSTM in 19.9% of complete windows.

Mean rolling Sharpe comparison

Portfolio	36-month mean	60-month mean
Authors GAN	3.638	3.199
Code 07 GAN	2.890	2.504
New LSTM	1.992	1.795
Transformer	1.467	1.228
50/50 blend	2.166	1.871

LSTM has the higher average, while the 36-month Transformer median is slightly higher.

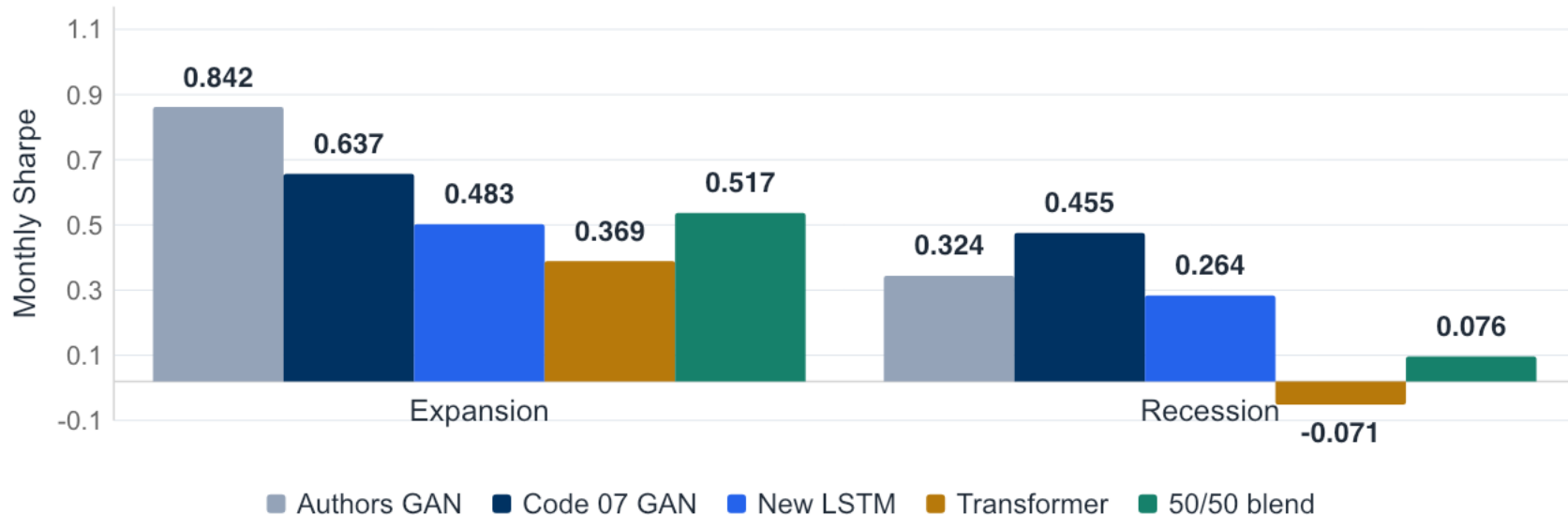
Economic regime definitions and sample sizes

Regime family	Definition	Test months
Business cycle	NBER expansion / recession	274 / 26
Market volatility	High / low previous-12-month volatility	123 / 177

The volatility cutoff comes from the training sample. Recession dates are retrospective.

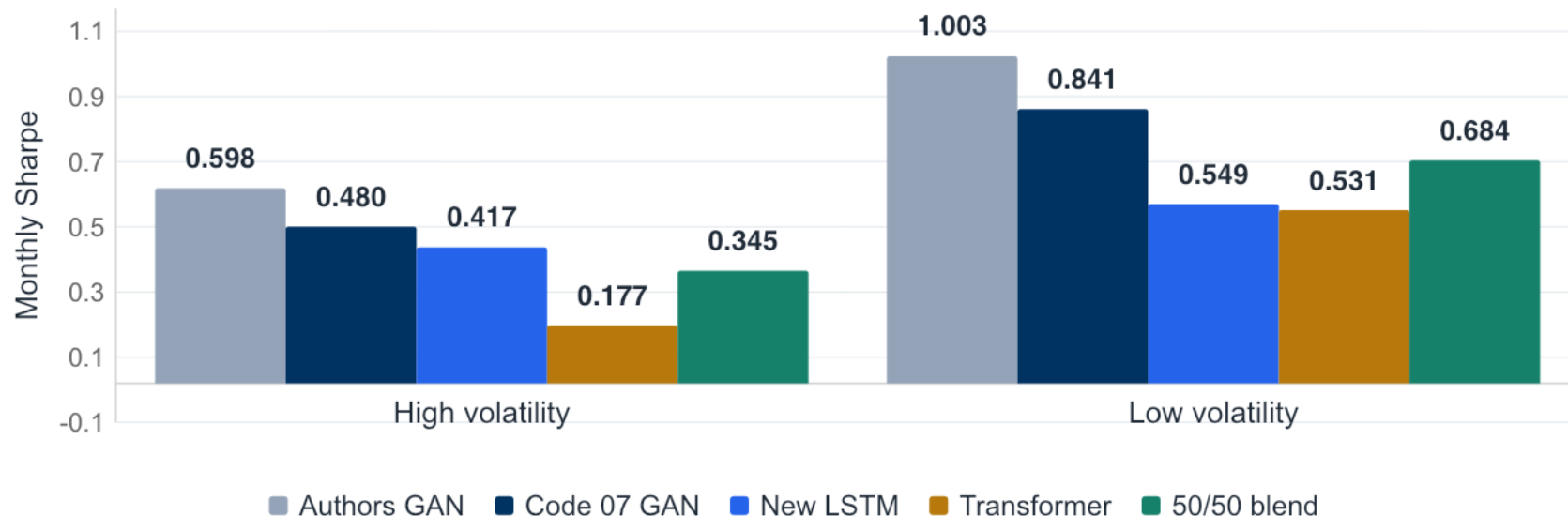
These are separate partitions. Recession estimates depend on only 26 months.

Business-cycle Sharpe comparison



New LSTM outperforms Transformer in both groups. Code 07 exceeds authors GAN in recession.

Market-volatility Sharpe comparison



New LSTM and Transformer are closest in low-volatility months.

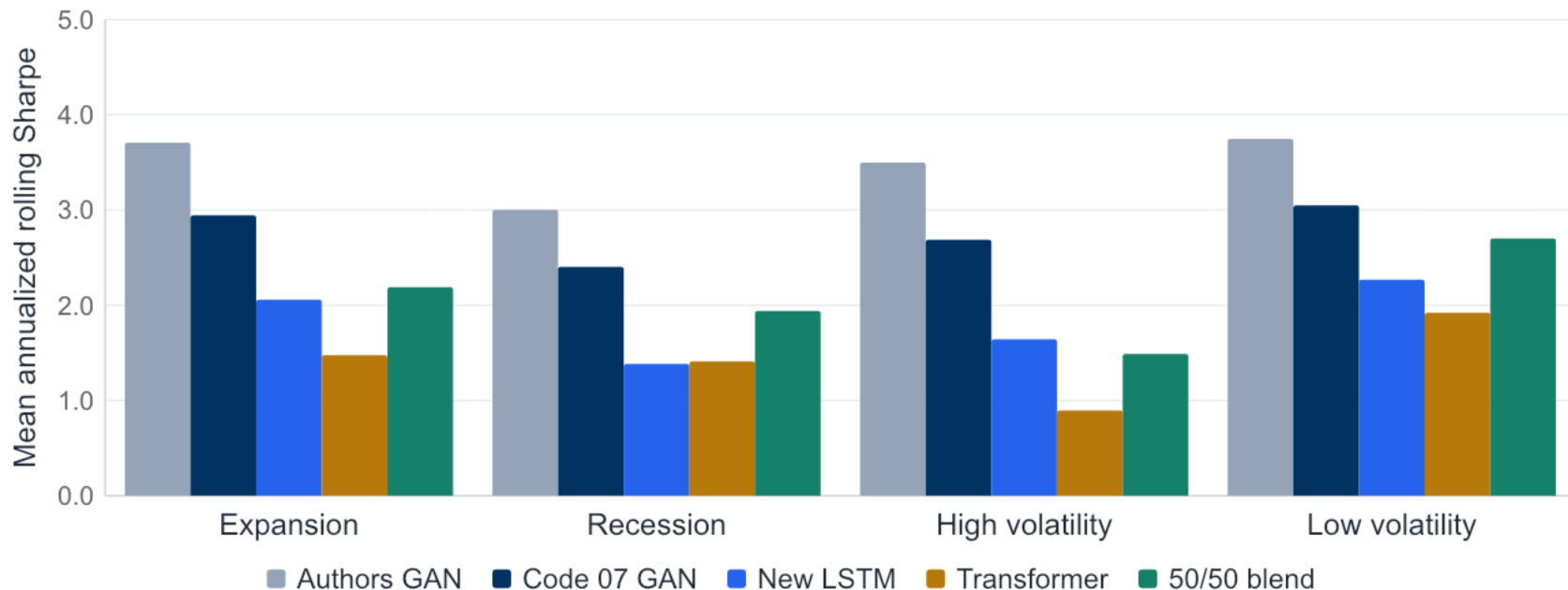
Conditional months and rolling endpoints

Analysis	Returns used	Question
Conditional regime Sharpe	Only labeled months	How does performance differ within the state?
Rolling by endpoint regime	All returns in each trailing window	What trailing performance do we observe in that state?

A 36-month recession-ending window can include many expansion months.

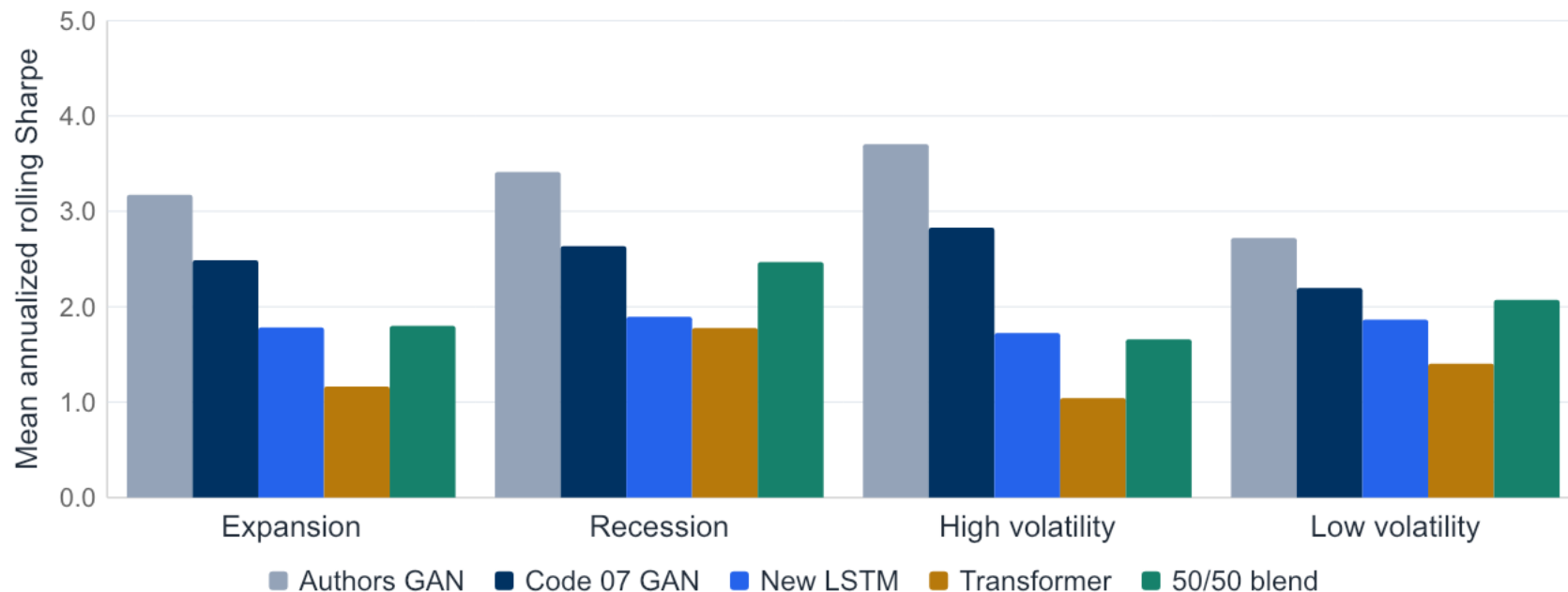
The two measures can rank the models differently because they use different returns.

36-month rolling performance by endpoint regime



In recession-ending windows, Transformer 1.409 slightly exceeds new LSTM 1.383.

60-month rolling performance by endpoint regime



In recession-ending windows, new LSTM 1.893 exceeds Transformer 1.775.

The fixed 50/50 architecture blend

Portfolio	Train	Validation	Test
LSTM	1.308	0.750	0.459
Transformer	1.049	0.589	0.304
50/50 blend	1.414	0.915	0.463

Test correlation between architectures is 0.292. Averaging lowers volatility.

Blend test Sharpe improves only slightly: 0.463 versus LSTM 0.459.

What the new comparisons show

Overall architecture result

New LSTM has higher full-test Sharpe and lower risk at the shared reduced budget.

Time and regime variation

Relative performance changes. Recession-ending windows differ from recession-only returns.

Combination result

A fixed blend helps in some groups, with only a small full-test Sharpe gain.

Limitations and the next experiment

The training budget limits the conclusion

Our completed 16/4/48 run does not reproduce the authors' full schedule or tuning search.

The evidence is descriptive

Windows overlap, recession samples are small, and test results have been examined previously.

A full GPU experiment is the next test

Benchmark one model, then train both architectures with nine seeds and refresh Codes 09–11.